

Interview Director

Why I built it, what changed, and what I still need to prove.

Part 1 - AI mock interview | Prepared by Pronita

I wanted the interview to hear the decisions behind the resume, not just ask someone to read it aloud.

Where I started

My starting point was a two-stage conversation: first, let the candidate introduce themselves; then ask them to explain a project they actually worked on. I wanted the second stage to uncover ownership, alternatives, constraints, and results rather than reward a polished but vague story.

Working on this introduced me to LiveKit and Tavus. The difficult part was not simply putting a talking face on a page. It was deciding who controls the conversation, what happens when someone interrupts, and which version of an answer should become evidence.

What changed as the project developed

The early interface was useful for trying the flow, but it also made the gap between a demo and a real interview obvious. A preset candidate name and a decorative avatar could give the wrong impression. I moved toward clearly separated live and credential-free paths, explicit connection states, and a more readable laptop layout.

I originally thought mainly about the questions an employer would want answered. Then I started asking what the candidate needs: a chance to hear a question again, captions that stay visible, enough time to think, and a way to correct a transcription mistake. Those became part of the design, not just finishing touches.

The scope I settled on

This is a mock-interview prototype with an evidence map, not a validated hiring assessment. It has two introduction questions and three experience questions, with no more than two targeted follow-ups for each primary question. The point is to collect a useful, reviewable account of someone's work, not to make an automated hiring decision.

Current boundary: the production integration code is present, but a complete credentialed voice-and-avatar session has not been documented as verified. This roadmap keeps that distinction visible.

The decisions that shaped it

Keep the workflow outside the language model

I chose a provider-independent state machine as the authority: briefing, introduction, experience, debrief, and ended. The language model can acknowledge an answer, but the Director owns the permitted questions and transitions. That makes timing and repeat-event behavior testable without relying on a model to follow a long prompt perfectly.

The introduction and experience agents share one LiveKit AgentSession. They have different stage instructions, while confirmed answers and evidence remain in shared state. Event IDs reject duplicate events; stage epochs prevent an old answer from being accepted after a handoff.

Use a time boundary without making every answer a countdown

I did not want a visible hard timer to make the conversation feel like a speed test. The interface uses gentle guidance, while the Director has deterministic stage fallbacks: 150 seconds for introduction and 300 seconds for experience. Question and follow-up limits also keep the interview from getting stuck.

Normal completion still depends on finishing the primary questions and collecting the required signals. All six experience categories are tracked, but the current normal-transition gate requires context, ownership, reasoning, and impact. Difficulty and reflection are tracked without being mandatory gates. That is a prototype policy choice I still need to review.

Give interviewer audio one owner

The live worker sends speech recognition through a confirmation gate, then generates a short response and TTS. With Tavus connected, the LiveKit Tavus plugin sends that speech to the avatar for synchronized audio and video in the same room. The direct agent audio output is disabled so the candidate does not hear two voices.

The standalone Tavus conversation endpoints are separate from the normal interview path. They should not be started alongside the plugin. If the avatar fails, the application can fall back to direct LiveKit speech, but that is an audio-only fallback - not a successful Tavus demonstration.

Treat interruptions as conversation, not failure

The worker is configured to yield speech when an interruption qualifies, with a 600 ms/two-word threshold and a diagnostic cooldown. The next acknowledgment should be brief instead of replaying the whole response. These settings are implemented, but a test of events cannot establish how well they work with real microphones, accents, background noise, and short interruptions.

What I learned from the edge cases

A transcript should not become evidence automatically

Speech recognition can get names, accents, and technical terms wrong. I added a confirm-or-edit step after final transcription. Interim text is ignored, and a correction is recorded without being treated as a negative signal. The Director only receives the confirmed answer.

This creates a real trade-off: confirmation adds time and interrupts the conversational rhythm. Stage timers also keep running, so a confirmation can become stale during a handoff. The code rejects it to protect stage state; the next usability pass needs to make that situation clearer and reconsider how review time should interact with the fallback.

Tab detection is not ChatGPT detection

I initially wanted a way to notice whether someone had switched away to use another tool. The browser can observe visibility, focus, and paste events, but it cannot reliably identify another tab's content or prove misconduct. I narrowed the feature to disclosed context signals and added Focus Assist to pause monitoring. I would not use those events as an automatic cheating verdict.

The evidence map needs careful language

The current extractor uses deterministic keyword patterns. That makes tests repeatable, but it can miss a well-explained answer or count a word that does not really demonstrate a skill. The coverage score is therefore a prototype summary, not a calibrated measure of candidate ability.

The interface includes a human-review request, but that button currently records a local UI state; it is not connected to a staffed review queue. Before presenting that as recourse, I need to build an actual delivery and tracking process.

What the current evidence supports

On 3 September 2026, a fresh local run passed 24 tests across four test files. The frontend and agent builds passed. The build still warns about a large frontend bundle, so code splitting belongs on the performance backlog.

Those results support the implemented state-machine and local integration behavior. They do not prove live voice quality, lip-sync, acoustic interruption handling, hosted reconnection, or room cleanup. Latency also needs careful interpretation: the browser uses an active-speaker observation as an audio-start proxy, and a trustworthy first-token measurement is not currently available.

My next milestones

1. Prove one full live session

Start the API, agent worker, and client with the same LiveKit project. Verify model access and the Tavus PAL/Face configuration. Complete a real introduction and experience interview, confirm the transcripts, and save redacted logs. I will call this milestone passed only when the actual avatar speaks, the normal handoff happens once, and the final evidence retains both stages.

2. Test the parts that are easy to demonstrate badly

Interrupt at least three times under different conditions. Run a separate rehearsal that waits the full 150 seconds, rather than relying only on a unit test of the timer. End the interview and confirm the Tavus participant leaves. Test denied microphone permission, a lost connection, and a late transcript confirmation. Record observed latency, including the confirmation delay.

3. Improve the candidate experience with real feedback

Ask a small number of volunteers to try the flow with informed consent. Check captions and controls on laptop screens, repeat-question behavior, and whether follow-ups feel useful or repetitive. Review false positives and missed evidence before changing the extractor. Make the displayed coverage score and human-review affordance match what the system can actually support.

4. Deploy only after the local live path is credible

Host the frontend, HTTPS token API, and always-on agent worker separately as needed. Then repeat the live checks on the hosted system. Before real applicant use, add authentication, consent, transcript retention and deletion, rate limiting, durable session handling, and provider spending controls. A successful build alone is not that milestone.

The finish line I am using: a repeatable, recorded live session with clear limitations - not just a smooth demo clip.

Implementation sources and useful links

[Repository - source, setup, architecture, and demo instructions](#)

[State machine reviewed at commit 14c0b1d](#)

[Live worker reviewed at commit 14c0b1d](#)

Technical references used in the project: LiveKit Agents, AgentSession documentation, and the LiveKit Tavus plugin guide. These are implementation references, not evidence that a live session passed.

[LiveKit Agents documentation](#)

[LiveKit Tavus plugin documentation](#)